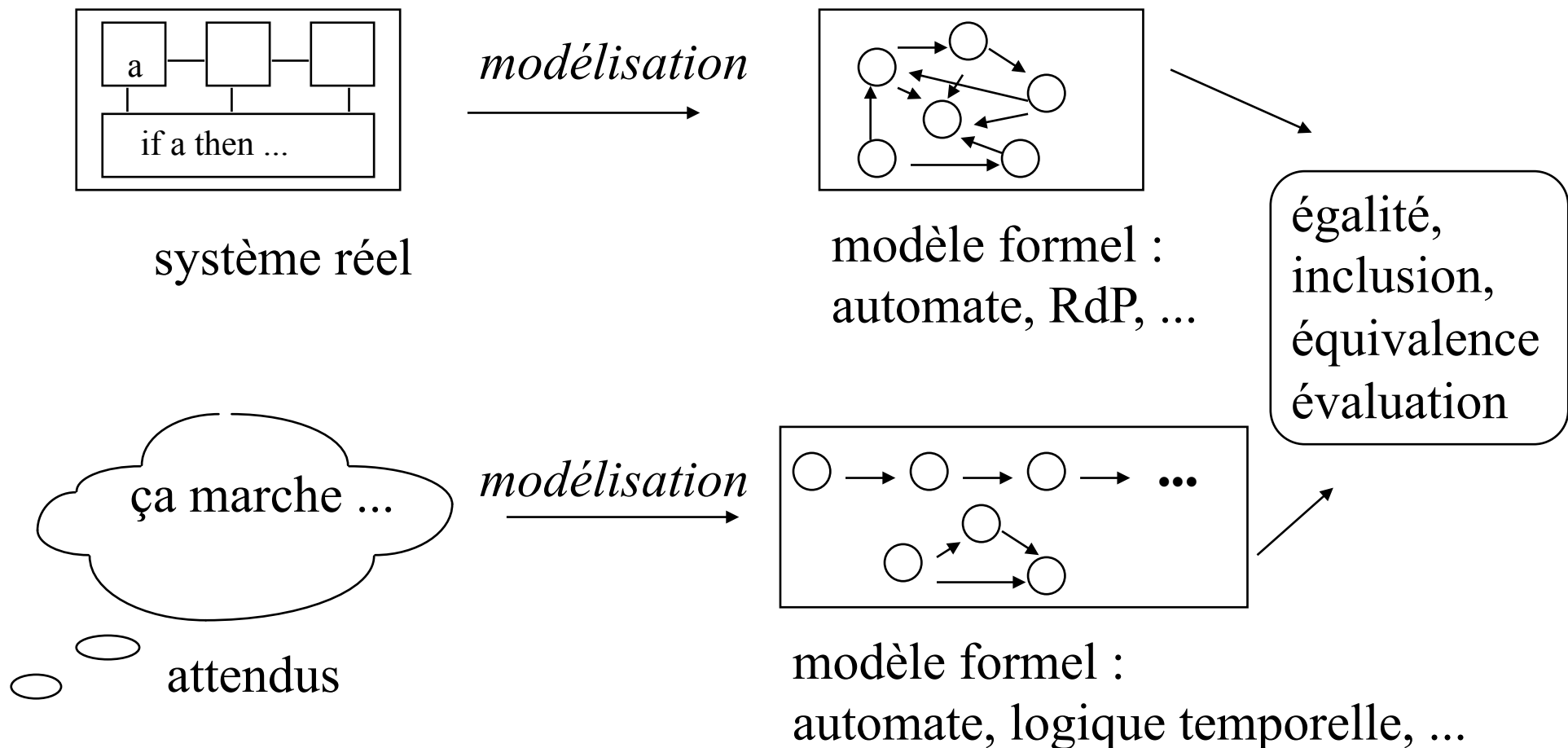


Vérification de propriétés CTL

- 1- Rappel: définition de CTL
- 2- Algorithmes élémentaires du model-checking
- 3- Utilisation de NuSMV (TP)

Processus de vérification



Modèle du système

Structure de Kripke = un système de transitions étiqueté sur états :
 $\langle S, T, S_0, AP, h \rangle$

S : ensemble fini d'états

T : relation de transition incluse dans $S \times S$

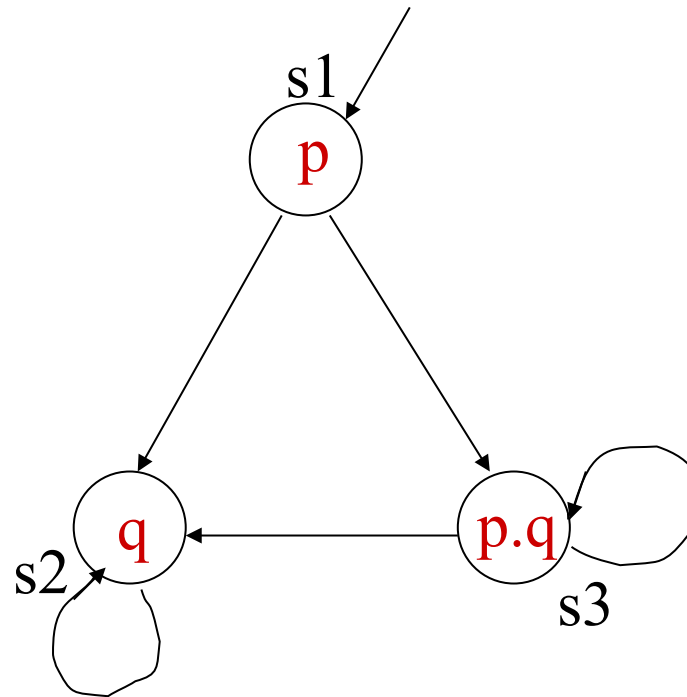
S_0 : ensemble des états initiaux

AP : ensemble des propositions atomiques

h : fonction de satisfaction : $S \rightarrow 2^{AP}$ qui associe à chaque état les propriétés atomiques vraies dans cet état.

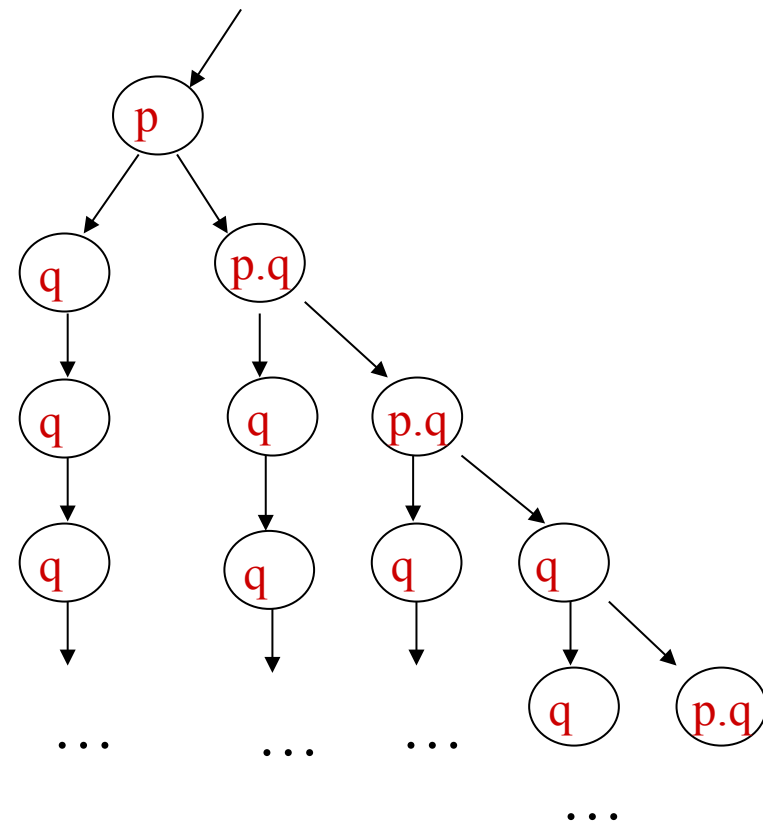
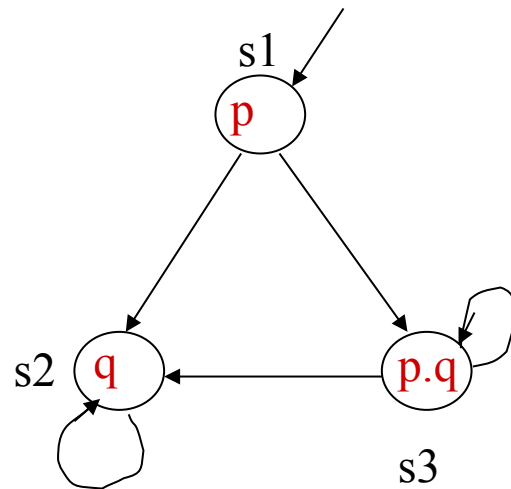
Une structure de Kripke

- $\langle S, T, So, AP, h \rangle$
 - $S = \{s1, s2, s3\}$
 - $T = \{(s1, s2), (s1, s3), (s2, s2), (s3, s2), (s3, s3)\}$
 - $So = \{s1\}$
 - $AP = \{p, q\}$
 - $h :$
 - $s1 \rightarrow p$
 - $s2 \rightarrow q$
 - $s3 \rightarrow p.q$



Arbre d'exécution infini

- Obtenu par *déroulement* de la structure de Kripke



Modèle de la spécification

Logique temporelle CTL :

Une formule CTL f est définie inductivement (f_1, f_2 sont des formules CTL, AP un ens. des prop atomiques) :

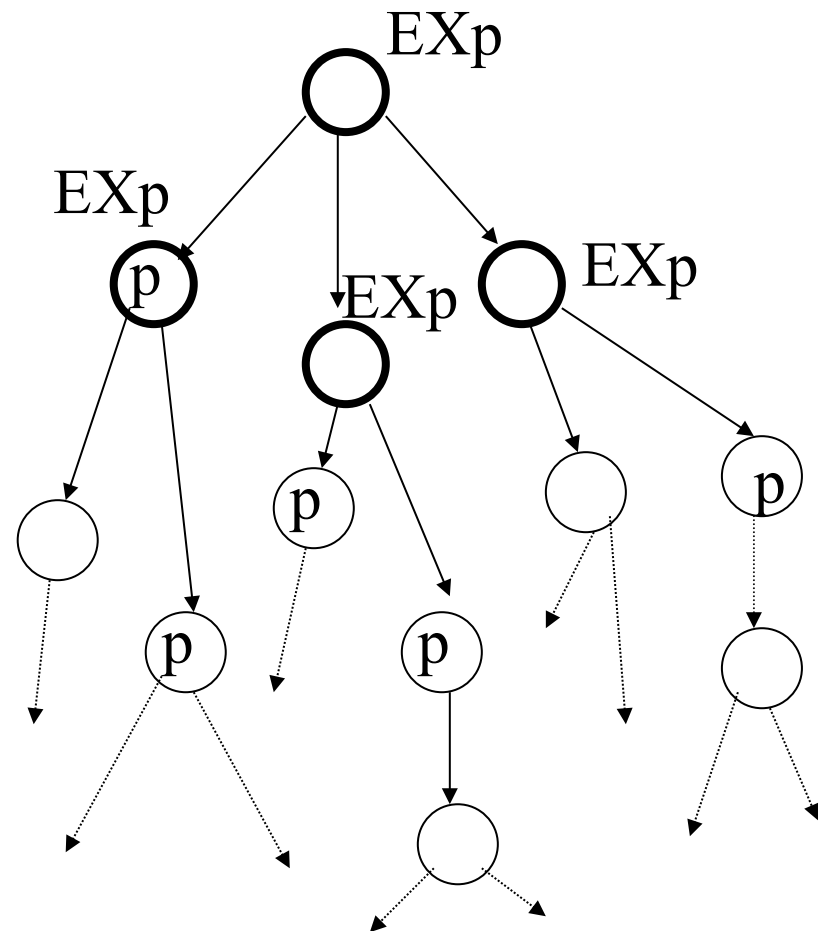
- une formule propositionnelle sur AP,
- $f_1 \text{ or } f_2, f_1 \text{ and } f_2, \text{not } f_1,$
- **EX** $f_1, \mathbf{AX} f_1,$
- **E** $f_1 \mathbf{U} f_2, \mathbf{A} f_1 \mathbf{U} f_2.$

Modèles de formule CTL : états de l'arbre d'exécution infini obtenu par déroulement de la structure de Kripke $K = \langle S, T, S_0, AP, h \rangle$

Opérateur EX

$s \models \text{EX } p$: il existe s' un successeur
de s tel que $s' \models p$

Etats vérifiant $\text{EX } p$

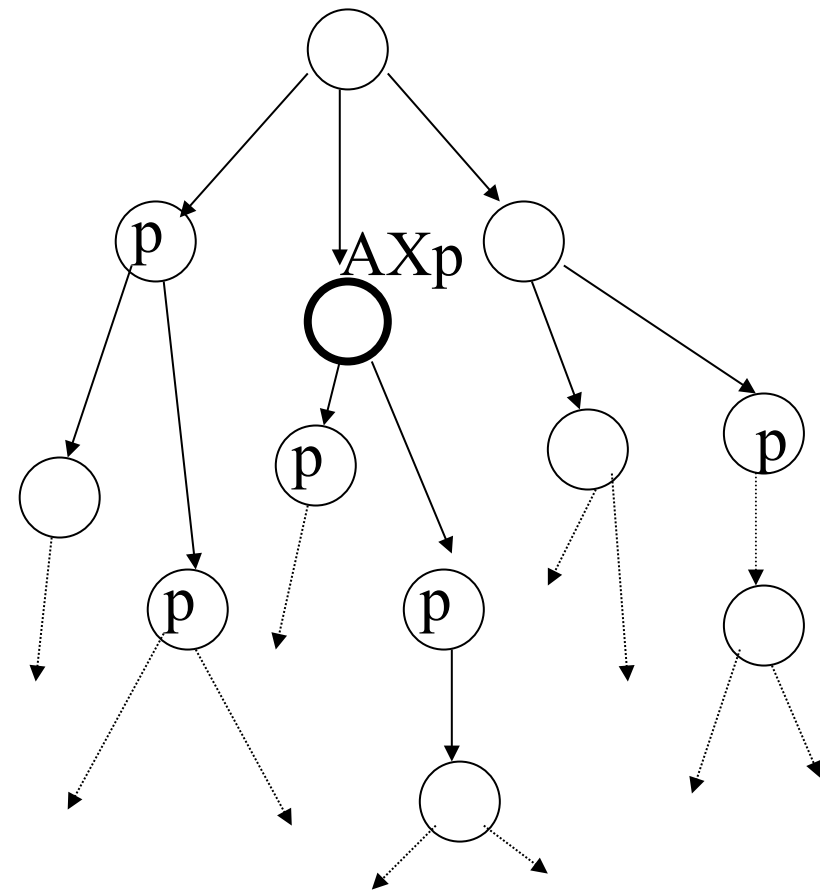


Rq : p est une proposition atomique ou une
sous-formule CTL

Opérateur AX

Etats vérifiant AXp

$s \models \text{AX } p$: pour tout successeur s'
de s , on a $s' \models p$

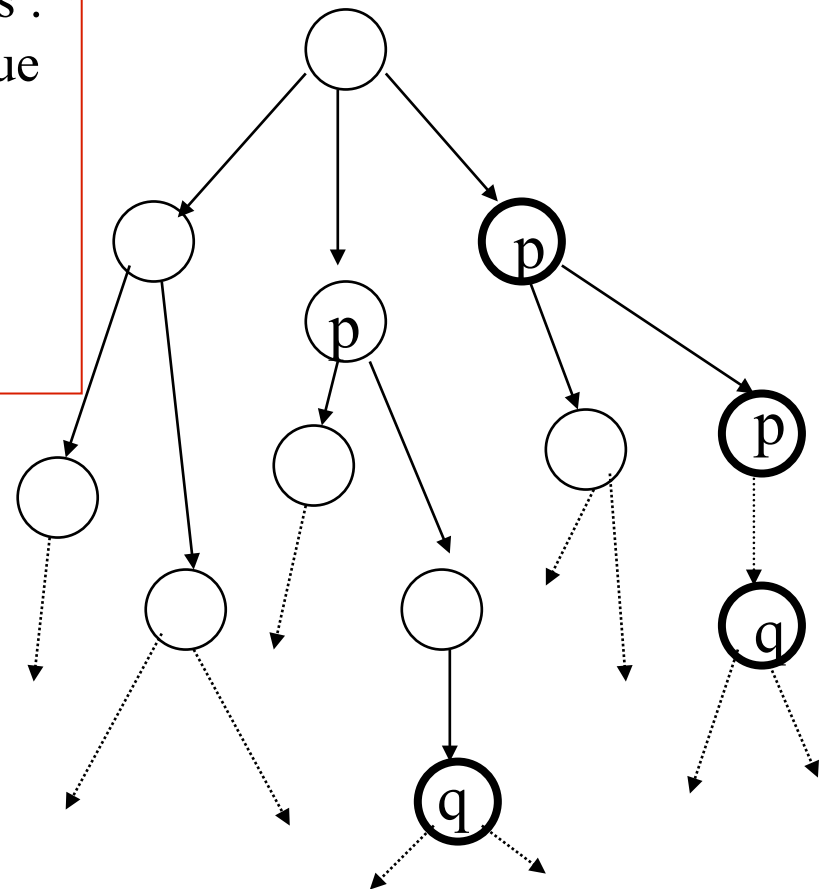


Opérateur EU

Etats vérifiant $E p U q$

$s \models E p U q$: il existe une séquence σ issue de s :
 $\sigma = s \rightarrow s' \dots \rightarrow s'' \rightarrow \dots$ telle que

- $s \models q$ ou alors
- $s \models p$ et il existe $s'' \models q$ et
quelquesoit s' sur σ entre s et s'' , $s' \models p$



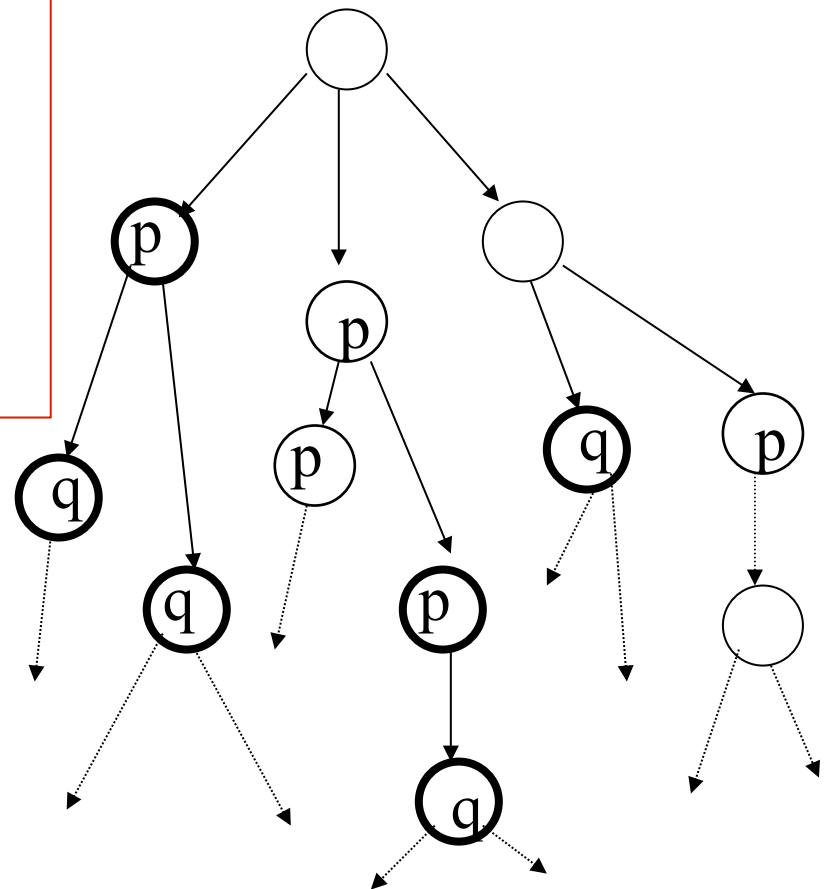
Opérateur AU

Etats vérifiant $A \ pUq$

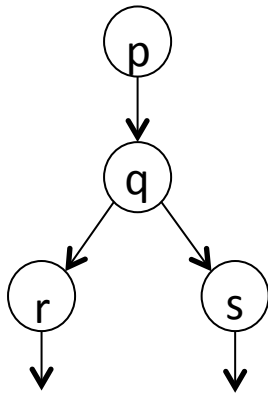
$s \models A \ p \ U \ q$: pour toute séquence σ issue de s :

$$\sigma = s \rightarrow s' \dots \rightarrow s'' \rightarrow \dots \text{ vérifie}$$

- $s \models q$ ou alors
- $s \models p$ et il existe $s'' \models q$ et
quelquesoit s' sur σ entre s et s'' , $s' \models p$

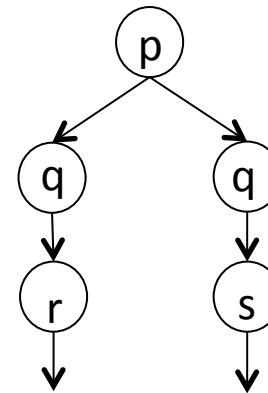


CTL permet de distinguer les points de branchement



« de p, on atteint **forcément** un état q, à partir duquel **il existe** un état successeur satisfaisant r et **il existe** un état successeur satisfaisant s »

$$p \wedge \mathbf{AX} (q \wedge (\mathbf{EX} r \wedge \mathbf{EX} s))$$



« de p, on **peut** atteindre un état satisfaisant q, dont **tous les successeurs** satisfont r et un autre état q dont **tous les successeurs** satisfont s »

$$p \wedge \mathbf{EX} (q \wedge \mathbf{AX} r) \wedge \mathbf{EX} (q \wedge \mathbf{AX} s)$$

D'autres opérateurs non primitifs

$$\mathbf{AF} \, p = \mathbf{A} \, (\text{True} \, \mathbf{U} \, p),$$

$$\mathbf{EF} \, p = \mathbf{E} \, (\text{True} \, \mathbf{U} \, p),$$

$$\mathbf{AG} \, p = \mathbf{A} \, (\neg \, \mathbf{F} \, (\neg \, p)) = \neg \, \mathbf{EF} \, (\neg \, p),$$

$$\mathbf{EG} \, p = \mathbf{E} \, (\neg \, \mathbf{F} \, (\neg \, p)) = \neg \, \mathbf{AF} \, (\neg \, p),$$

Sémantique de CTL

Rq : p est une proposition atomique ou et f_1 et f_2 sont 2 formules CTL

$s \models p \Leftrightarrow p$ est vrai dans l'état s

$s \models f_1 \vee f_2 \Leftrightarrow s \models f_1$ ou $s \models f_2$

$s \models f_1 \wedge f_2 \Leftrightarrow s \models f_1$ et $s \models f_2$

$s \models \neg f_1 \Leftrightarrow s \not\models f_1$

$s \models EX f_1 \Leftrightarrow \exists s' \text{ tq } s \rightarrow s' \text{ et } s' \models f_1$

$s \models AX f_1 \Leftrightarrow \forall s' \text{ tq } s \rightarrow s' \text{ et } s' \models f_1$

$s \models EF f_1 \Leftrightarrow \exists \sigma = s \rightarrow s' \rightarrow \dots s'' \dots \rightarrow \dots \text{ et } s'' \models f_1$

$s \models AF f_1 \Leftrightarrow \forall \sigma = s \rightarrow s' \rightarrow \dots s'' \dots \rightarrow \dots \text{ et } s'' \models f_1$

$s \models EG f_1 \Leftrightarrow \exists \sigma = s \rightarrow s' \rightarrow \dots s^k \dots \rightarrow \dots \text{ et } \forall k \geq 0, s^k \models f_1 \text{ (et } s = s^0)$

$s \models AG f_1 \Leftrightarrow \forall \sigma = s \rightarrow s' \rightarrow \dots s^k \dots \rightarrow \dots \text{ et } \forall k \geq 0, s^k \models f_1 \text{ (et } s = s^0)$

$s \models E f_1 U f_2 \Leftrightarrow \exists \sigma = s \rightarrow s^1 \rightarrow \dots s^k \dots \rightarrow \dots \text{ et } \exists k \text{ tq } s^k \models f_2 \text{ et } \forall k > j \geq 0, s^j \models f_1 \text{ (et } s = s^0)$

$s \models A f_1 U f_2 \Leftrightarrow \forall \sigma = s \rightarrow s^1 \rightarrow \dots s^k \dots \rightarrow \dots \text{ et } \exists k \text{ tq } s^k \models f_2 \text{ et } \forall k > j \geq 0, s^j \models f_1 \text{ (et } s = s^0)$ ₁₃

Réécriture

On peut définir tous les opérateurs à partir de **EX**, **EG** et **EU** :

$$\mathbf{EF}(p) = \text{True} \mathbf{U} p$$

$$\mathbf{AX}(p) = \neg \mathbf{EX} (\neg p)$$

$$\mathbf{AF}(p) = \neg \mathbf{EG} (\neg p)$$

$$\mathbf{AG}(p) = \neg \mathbf{E} \text{ True } \mathbf{U} \neg p$$

$$\mathbf{A} p \mathbf{U} q = \neg \mathbf{E} \neg q \mathbf{U} (\neg p \wedge \neg q) \wedge \neg \mathbf{EG} (\neg q)$$

Définitions expansives des opérateurs **EF**, **EG** et **EU** :

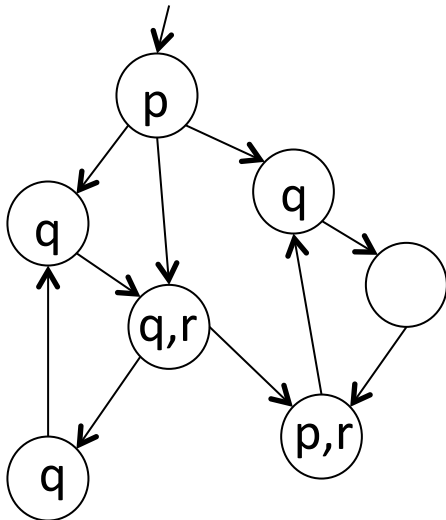
$$\mathbf{EF} p = p \vee \mathbf{EX} \mathbf{EF}(p)$$

$$\mathbf{EG} p = p \wedge \mathbf{EX} \mathbf{EG}(p)$$

$$\mathbf{E} p \mathbf{U} q = q \vee (p \wedge \mathbf{EX} (\mathbf{E} p \mathbf{U} q))$$

Model-checking de propriété CTL

- Soit M un système décrit par une structure de Kripke et φ une propriété CTL;
on dit que $M \models \varphi$ ssi s_0 l'état initial de S , satisfait $\varphi : s_0 \models \varphi$



A-t-on $M \models \varphi$?

$$\varphi = p \wedge EX q$$

$$\varphi = p \wedge AX q$$

$$\varphi = EG q$$

$$\varphi = r \Rightarrow EX EG q$$

$$\varphi = AX (EG q)$$

$$\varphi = AG (r \Rightarrow A (p U q))$$

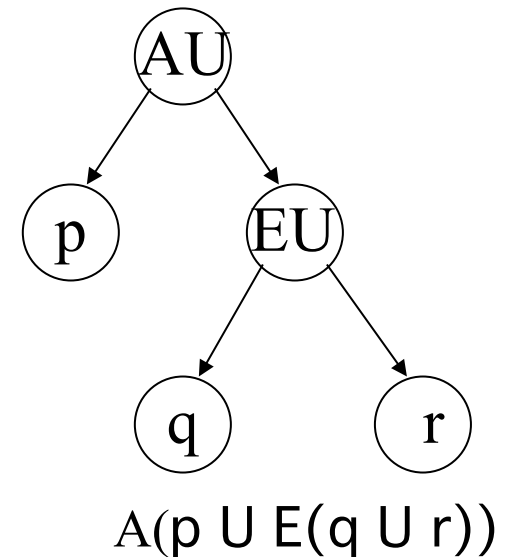
$$\varphi = AG EF q$$

$$\varphi = AG AF q$$

Vérification de propriétés CTL

- Soit M un système décrit par une structure de Kripke et φ une propriété CTL;
on dit que $M \models \varphi$ ssi s_0 l'état initial de S , satisfait $\varphi : s_0 \models \varphi$

1. Construire l'arbre syntaxique de la formule φ
 - nœuds : opérateurs CTL
 - feuilles : propriétés atomiques
2. Réécrire l'arbre en utilisant uniquement les opérateurs EX, AX, EU, AU
3. Parcourir l'arbre et pour chaque nœud opérateur N , appliquer la procédure de marquage de la structure de Kripke adaptée (Algo1, Algo2, Algo3 ou Algo4 ci-après).
4. Si s_0 est marqué comme satisfaisant φ , alors $M \models \varphi$



Vérification de EXp et AXp

Algo 1 : EX p

Pour chaque état e faire

Si $\exists$ e' un successeur de e tel $e' \models p$ Alors

$e \models \text{EX } p$

Sinon

$e \models \text{NOT EX } p$

Algo 2 : AX p

Pour chaque état e faire

Si $\forall$ e' un successeur de e , $e' \models p$ Alors

$e \models \text{AX } p$

Sinon

$e \models \text{NOT AX } p$

Vérification de $E p U q$ (1)

Algo 3 : $E p U q$

Début

$\forall e$ Démarquer(e)

$\forall e$ Vérifier_EU(e)

$\forall e$ Si e est non marqué Alors $e \models \text{NOT } E p U q$

Fin

Procédure Vérifier_EU(e)

Si e est non marqué Alors

 Si $e \models q$ Alors

 Marquer(e)

$e \models E p U q$

$\forall e'$ un prédécesseur de e , Propager_EU(e')

 Sinon si $e \models \text{NOT } p$ Alors

 Marquer(e)

$e \models \text{NOT } E p U q$

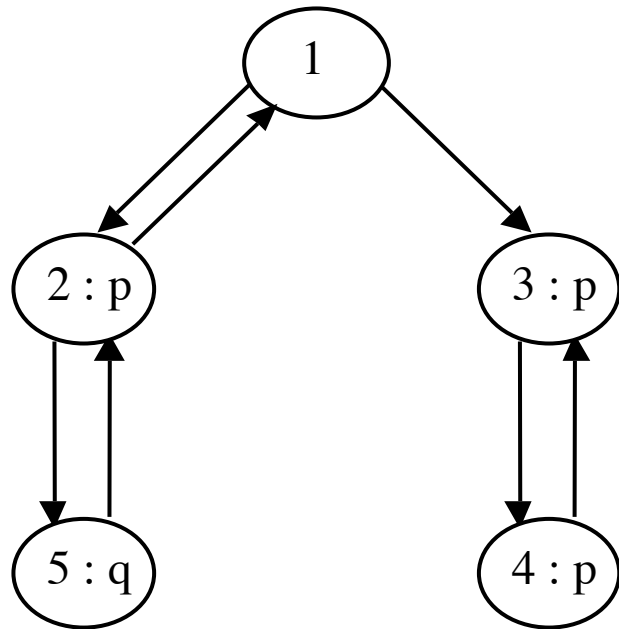
 Finsi

Finsi

Vérification de $\exists p U q$ (2)

```
Procédure Propager_EU(e)
Si e est non marqué Alors
  Marquer(e)
  Si  $e \models q$  ou  $e \models p$  Alors
     $e \models E p U q$ 
     $\forall e'$  un prédécesseur de  $e$  , Propager_EU( $e'$ )
  Sinon
     $e \models \text{NOT } E p U q$ 
  Finsi
Finsi
```

Vérification de E pUq - exemple



Vérification de $A \text{ p } U \text{ q}$ (1)

Algo 4 : $A \text{ p } U \text{ q}$

Début

$\forall e$ Démarquer(e)

$\forall e$ Vérifier_AU(e)

Fin

Procédure Vérifier_AU(e)

Si e est marqué Alors

 Si $e \models A \text{ p } U \text{ q}$ Alors

 Retour(Vrai)

 Sinon

 Retour(Faux)

 Finsi

sinon ...

Vérification de $A \vee B$ (2)

...

Sinon

Marquer(e)

Si $e \models q$ Alors

$e \models A \vee q$; Retour(Vrai)

Sinon

Si $e \models \text{NOT } p$ Alors

$e \models \text{NOT } A \vee q$; Retour(Faux)

Sinon

Si $\forall e'$ successeur de e , Vérifier_ $A \vee B$ (e') Alors

$e \models A \vee q$; Retour(Vrai)

Sinon

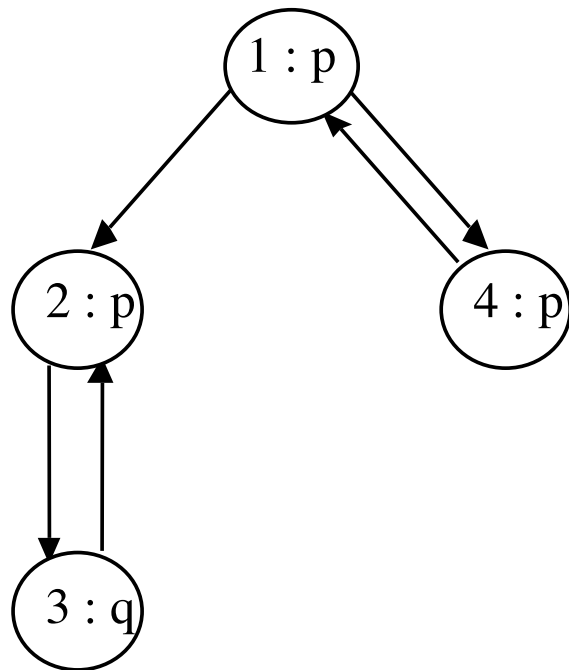
$e \models \text{NOT } A \vee q$; Retour(Faux)

Finsi

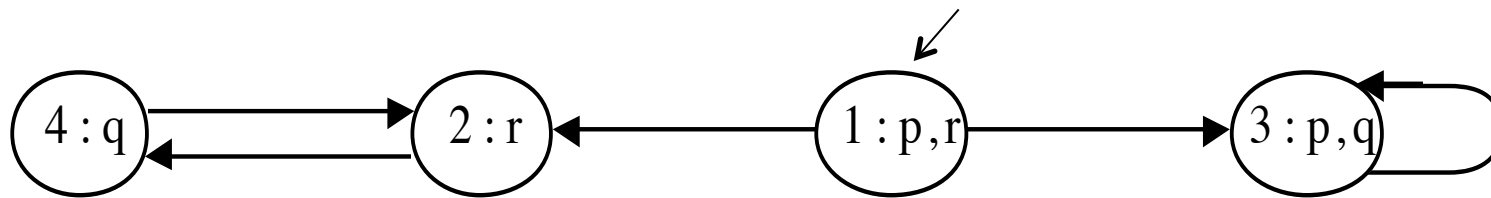
Finsi

Finsi

Vérification de $A_p \cup q$ - exemple



Vérification d'une formule avec opérateurs imbriqués - exemple



Evaluation de $A p \cup (E q \cup r)$

Conclusion

- Complexité de l'algorithme : $O(|S| \times |\varphi|)$
- Utilisable sur des systèmes à 10^6 - 10^7 états (20 bits de registres)
- Améliorations
 - Manipulation *d'ensembles* d'états (model-checking symbolique) (100 bits)
 - Restriction de la profondeur des traces d'exécution analysées et utilisation de SAT/SMT solvers (1000-10000 bits selon la propriété)
 - Abstractions :
 - Identifier une relation d'équivalence entre états et appliquer le model-checking sur le graphe quotient
 - Construction de la relation d'équivalence, préservant les propriétés intéressantes (pas tjs automatique)
 - Raisonnement compositionnel
 - Modèle structuré en composants, disposant de propriétés locales (vérifiables sur chaque composant)
 - Vérifier des propriétés globales par assemblage des propriétés locales (pas tjs possible)

Des parcours plus efficaces

Manipulation unitaire d'ensembles d'états étiquetés par la même sous-formule plutôt que des états pris individuellement.

On note S_φ l'ensemble des états de la structure de Kripke satisfaisant φ

On note $\text{PRE}(X)$ l'ensemble des états prédécesseurs en une transition d'un ensemble d'états X

$$\text{PRE}(X) = \{ e \mid \exists e' : e \rightarrow e' \text{ et } e' \in X \}$$

Quelques éléments sur les points fixes

[Tarski 50]

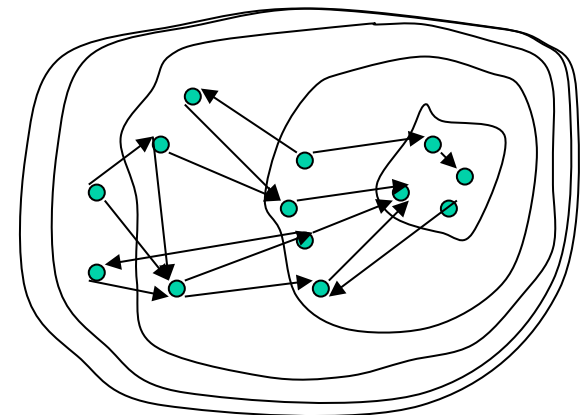
- Soient Q un ensemble, 2^Q l'ensemble des parties de cet ensemble
- Soit $f : 2^Q \rightarrow 2^Q$
 f est monotone si : X et $Y \subseteq 2^Q$ tq $X \subseteq Y$ alors $f(X) \subseteq f(Y)$
 f est $\cup$ -continue (resp. $\cap$ -continue) si $\cup f(X_i) = f(\cup X_i)$ (resp. $\cap f(X_i) = f(\cap X_i)$)
- Si f est monotone et continue, alors
 - $\cup f^i(\emptyset)$ est son plus petit point fixe, majoré par Q
 - $\cap f^i(\emptyset)$ est son plus grand point fixe, minoré par $\emptyset$ [Kleene]
- Pour $Y \subseteq Q$ et $F : 2^Q \rightarrow 2^Q$ continue,
 - PRE
 - $X \rightarrow Y \cup F(X)$
 - $X \rightarrow Y \cap F(X)$ sont continues : leur solution est leur point fixe

Calculs de point-fixe des opérateurs CTL (1)

- $S_{EX\ p} = \{e \mid e \rightarrow e' \text{ et } e' \models p\} = \text{PRE}(S_p)$
- $S_{EF\ p} = \{e \mid e \models p \text{ ou } e \rightarrow e' \dots \rightarrow e^n \rightarrow \dots \text{ et } e^n \models p\}$
 $= \{e \mid e \models p \text{ ou } e \rightarrow e' \text{ et } e' \models EFp\}$
 $= \{e \mid e \models p\} \cup \{e \mid e \rightarrow e' \text{ et } e' \models EFp\}$
 $= S_p \cup \text{PRE}(S_{EF\ p})$

Suite monotone croissante convergeant vers son plus petit point fixe

(On écrit aussi : $S_{EF\ p} = \mu. X = p \cup \text{PRE}(X)$)



Calcul du point-fixe de EF p

- $$S_{EF\ p} = \underbrace{\{p\}}_{S_{EF^0}} \cup \underbrace{PRE\{p\}}_{S_{EF^1}} \cup \underbrace{PRE\ PRE\{p\} \dots \cup PRE^n\{p\} \dots}_{S_{EF^n}} \cup \dots$$

- $$S_{EF^i\ p} = S_p \cup PRE(S_{EF^{i-1}\ p}) \text{ si } S_{EF^i\ p} = S_{EF^{i-1}\ p}, \text{ le point fixe est atteint}$$

```

Calcul_point_fixe_EF(p,y) /* initialisé avec y = ∅ */
    y' ← {p} ∪ PRE(y)
    si y' = y alors retourner y
    sinon retourner Calcul_point_fixe_EF(p,y')
fin
    
```

Calcul de point-fixe des opérateurs CTL (2)

- $$\begin{aligned}
 S_{EG\ p} &= \{e \mid e \models p \text{ et } e \rightarrow e' \dots \rightarrow e^n \rightarrow \dots \text{ et } e' \models p \text{ et } e^n \models p\} \\
 &= \{e \mid e \models p \text{ et } e \rightarrow e' \text{ et } e' \models EGp\} \\
 &= \{e \mid e \models p\} \cap \{e \mid e \rightarrow e' \text{ et } e' \models EGp\} \\
 &= S_p \cap PRE(S_{EG\ p})
 \end{aligned}$$

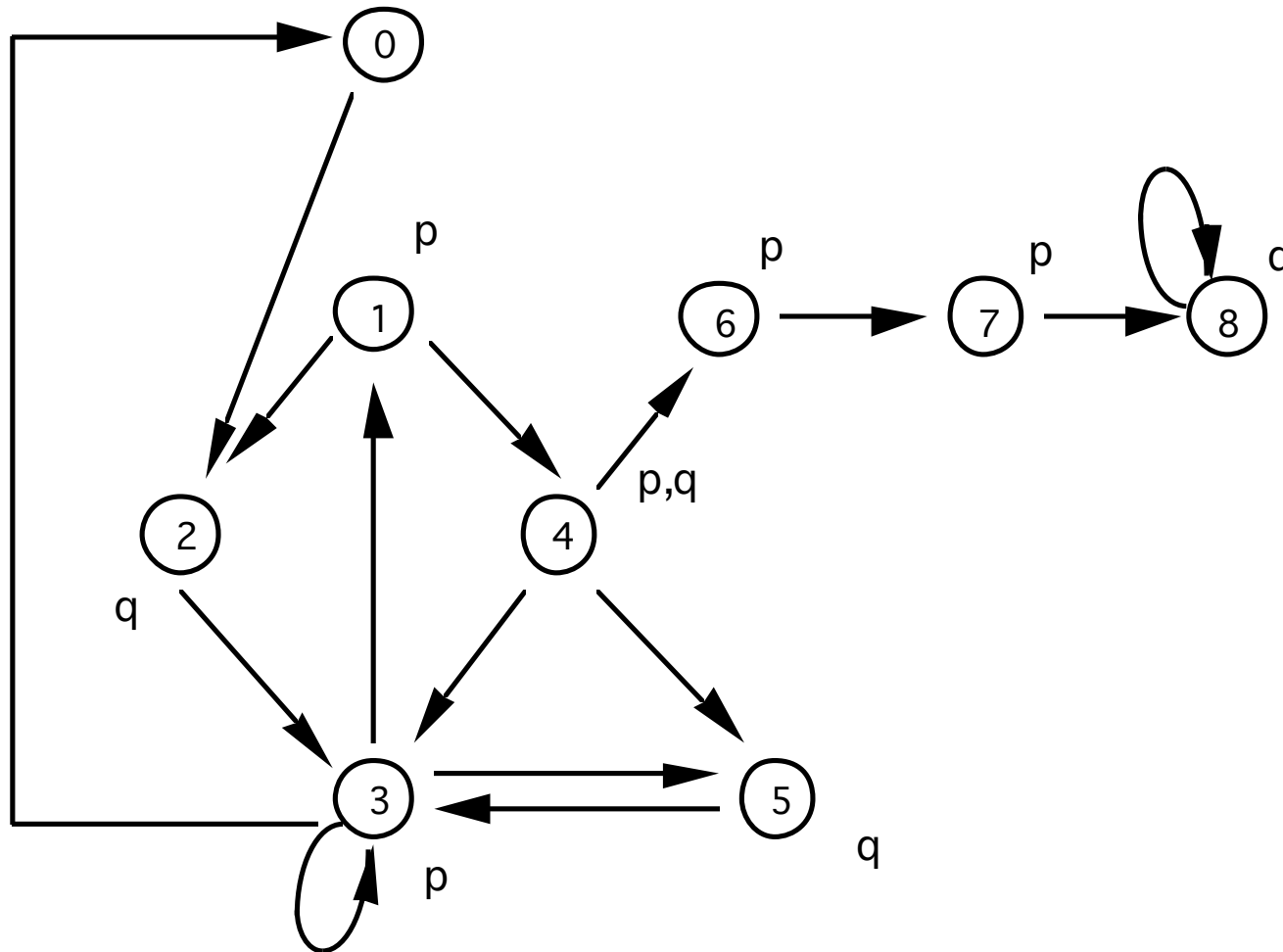
Suite monotone décroissante convergeant vers son plus grand point fixe

- $$\begin{aligned}
 S_{E\ p\ U\ q} &= \{e \mid e \models q \text{ ou } e \models p \text{ et } e \rightarrow e' \dots \rightarrow e^n \rightarrow \dots \\
 &\quad \text{et } \exists j \text{ tq } e^j \models q \ \forall i < j, e^i \models p\} \\
 &= \{e \mid e \models q \text{ ou } e \models p \text{ et } e \rightarrow e' \text{ et } e' \models E\ p\ U\ q\} \\
 &= \{e \mid e \models q\} \cup \{e \mid e \models p \text{ et } e \rightarrow e' \text{ et } e' \models E\ p\ U\ q\} \\
 &= S_q \cup (S_p \cap PRE(S_{E\ p\ U\ q}))
 \end{aligned}$$

Suite monotone croissante convergeant vers son plus petit point fixe

Calcul par point-fixe - exemple

Déterminez S_{EFp} , S_{EGp} et S_{EpUq} par calcul de point-fixe



Model-Checker symbolique (1)

Formule CTL

function Eval(φ)

case

φ : prop atomique $p \Rightarrow$ retourner S_p
 $\varphi = \neg q \Rightarrow$ retourner **complement**(**Eval**(q))
 $\varphi = r \vee s \Rightarrow$ retourner **Eval**(r) $\cup$ **Eval**(s)
 $\varphi = \mathbf{EX}q \Rightarrow$ retourner **EvalEX**(**Eval**(q))
 $\varphi = \mathbf{EG}q \Rightarrow$ retourner **EvalEG**(**Eval**(q), **true**)
 $\varphi = \mathbf{EU}(r, s) \Rightarrow$ retourner **EvalEU**(**Eval**(r), **Eval**(s), **false**)

fin case

fin

Ens des états vérifiant φ

function EvalEX(S_φ)

retourner **PRE**(S_φ)

fin

Ens des états vérifiant $\mathbf{EX}\varphi$

Model-Checker symbolique (2)

*Ens des états vérifiant $EG\varphi$
à l'itération courante*

```
function EvalEG( $S_\varphi, y$ )  
   $y' \leftarrow S_\varphi \cap \mathbf{EvalEX}(y)$   
  si  $y' = y$  alors  
    retourner  $y$   
  sinon  
    retourner EvalEG( $S_\varphi, y'$ )  
fin
```

*Ens des états vérifiant $E\varphi U\psi$
à l'itération courante*

```
function EvalEU( $S_\varphi, S_\psi, y$ )  
   $y' \leftarrow S_\psi \cup (S_\varphi \cap \mathbf{EvalEX}(y))$   
  si  $y' = y$  alors  
    retourner  $y$   
  sinon  
    retourner EvalEU( $S_\varphi, S_\psi, y'$ )  
fin
```

Model-checking symbolique de propriété CTL

$$M \models \varphi \text{ ssi } s_0 \in \mathbf{Eval}(\varphi)$$

Model Checker NuSMV

- Successeur de SMV, développé par CMU en 1995, intégrant les différentes techniques développées sur les dernières décennies :
 - LTL et CTL,
 - Bounded Model-Checking
 - Algorithmes énumératif, BDD, SAT, équités
- <http://nusmv.fbk.eu>